Abroad Education Counselling and Student Application Management System

Project / Subject	Database Management Systems
Tasks covered	Database implementation; NoSQL queries and transactions
Database / Stack	Apache CouchDB + Django / Django REST Framework (planned backend)
Student / Team	

1. Database Implementation

1.1 Environment and Database Creation

- **Local Environment Setup:** Apache CouchDB was launched locally, and the Fauxton web interface was accessed via a web browser at `http://127.0.0.1:5984/_utils/`.
- **Authentication & Security:** Administrator credentials were authenticated securely without exposing passwords, API keys, or sensitive credentials in the documentation.
- **Database Creation:** A database named `counselling_db` was created.

Data Modeling Guidelines: Lowercase values were defined for document types alongside stable, predictable document identifiers. References across entities were implemented directly via document IDs rather than SQL relational constraints.

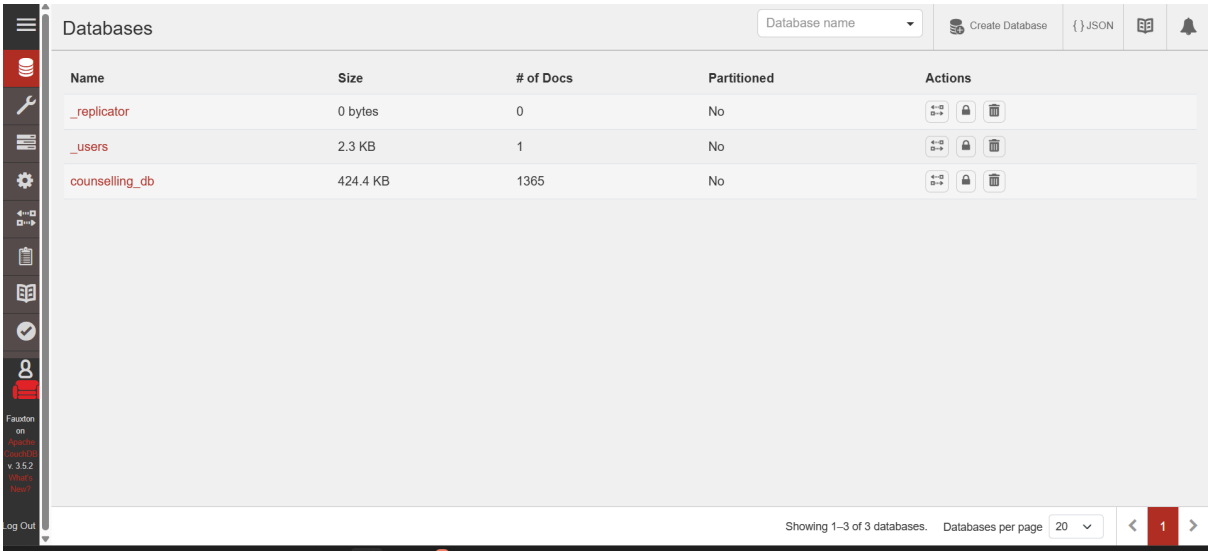


Figure 1: Apache CouchDB Fauxton Web Interface displaying active databases including counselling_db, illustrating environment setup and successful database creation.

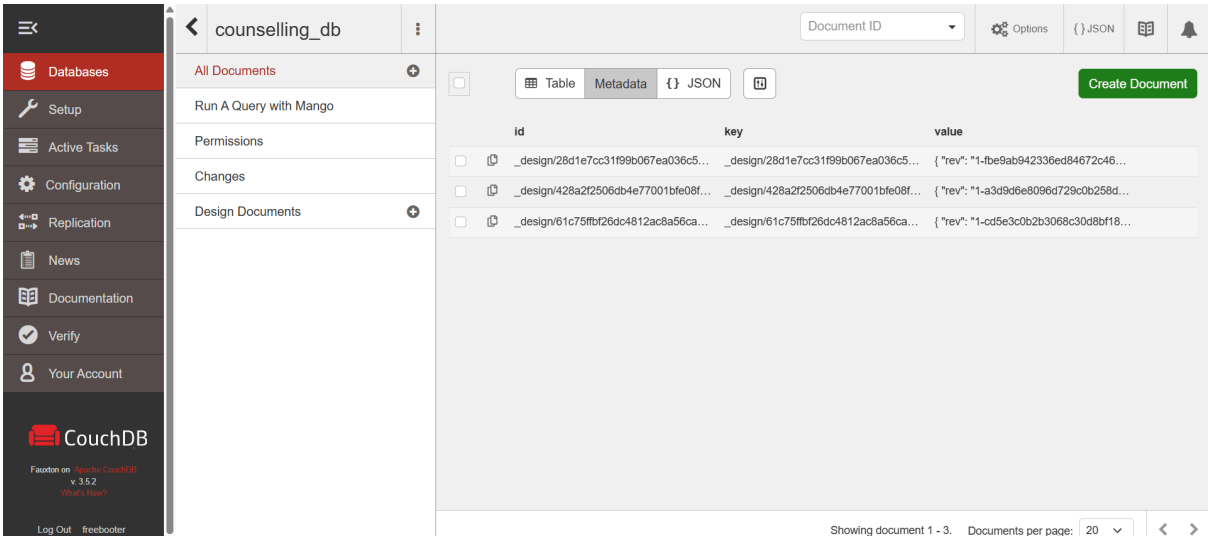


Figure 2: Fauxton interface showing the document list and design documents within counselling_db, verifying internal database structure and index initialization.

1.2 Document Design and Sample Data

Representative sample documents were designed and created in the CouchDB database using fictional data. Consistent references were maintained across documents (e.g., `application.student_id` referencing the student, `application.university_id` referencing the university, etc.), and CouchDB document revisions (`_rev`) were recorded after saving.

Student Document (`student_001`):

```JSON
{
  "_id": "student_001",
  "_rev": "1-7c91e4a112233445566778899aabbccd",
  "type": "student",
  "first_name": "John",
  "last_name": "Doe",
  "email": "john.doe@example.com",
  "phone": "+1234567890",
  "assigned_counsellor_id": "counsellor_001",
  "created_at": "2026-10-01T10:00:00Z"
}
```

Counsellor Document (`counsellor_001`):

```JSON
{
  "_id": "counsellor_001",
  "_rev": "1-8d92f5b2233445566778899aabbccdde",
  "type": "counsellor",
  "first_name": "Jane",
  "last_name": "Smith",
  "email": "jane.smith@counsellingagency.com",
  "department": "Overseas Admissions",
  "created_at": "2026-09-15T08:30:00Z"
}
```

University Document (`university_001`):

```JSON
{
  "_id": "university_001",
  "_rev": "1-9e03a6c33445566778899aabbccddeef",
```

```
  "type": "university",
  "name": "Global Tech University",
  "country": "United States",
  "website": "https://www.globaltech.edu",
  "created_at": "2026-09-01T12:00:00Z"
}
```

Course Document (`course_001`):

```JSON
{
  "_id": "course_001",
  "_rev": "1-0f14b7d445566778899aabbccddeeff0",
  "type": "course",
  "university_id": "university_001",
  "title": "M.S. in Computer Science",
  "duration_months": 24,
  "tuition_fee_usd": 45000,
  "created_at": "2026-09-05T14:20:00Z"
}
```

Application Document (`application_001`):

```JSON
{
  "_id": "application_001",
  "_rev": "1-1a25c8e5566778899aabbccddeeff001",
  "type": "application",
  "student_id": "student_001",
  "university_id": "university_001",
  "course_id": "course_001",
  "counsellor_id": "counsellor_001",
  "status": "submitted",
  "submitted_at": "2026-10-05T11:15:00Z"
}
```

Screenshots

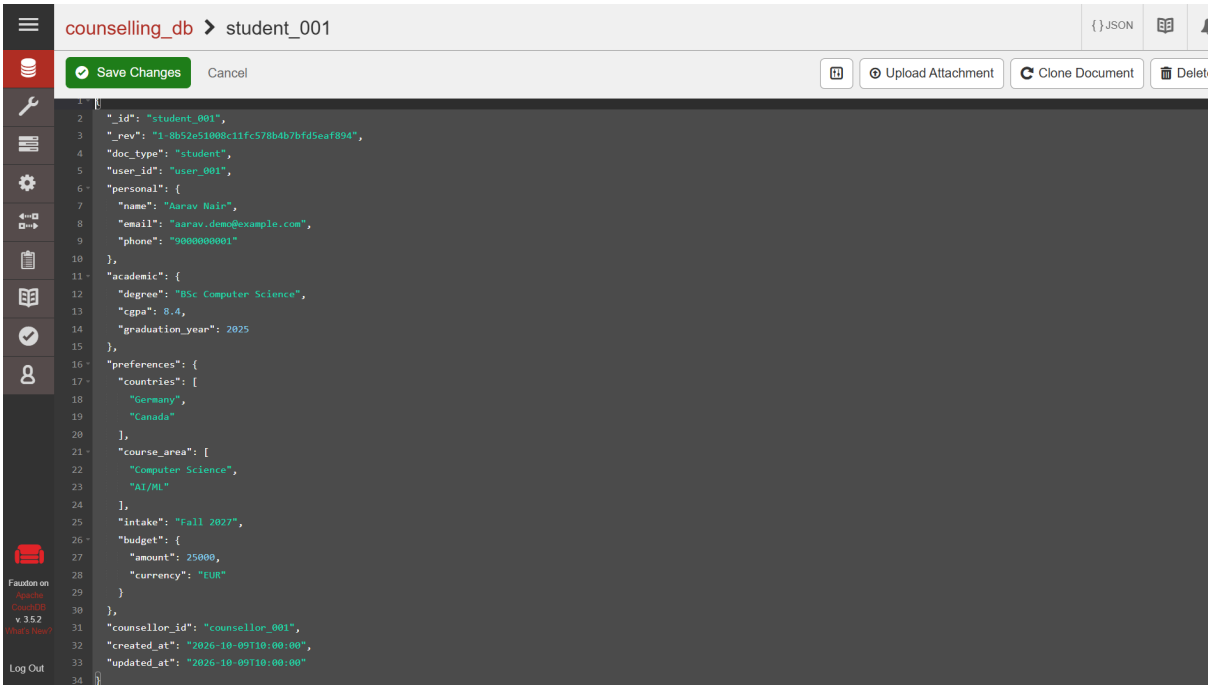


Figure 3: Detailed view of student_001 document editor in Fauxton illustrating JSON document fields, personal/academic profiles, and revision metadata.

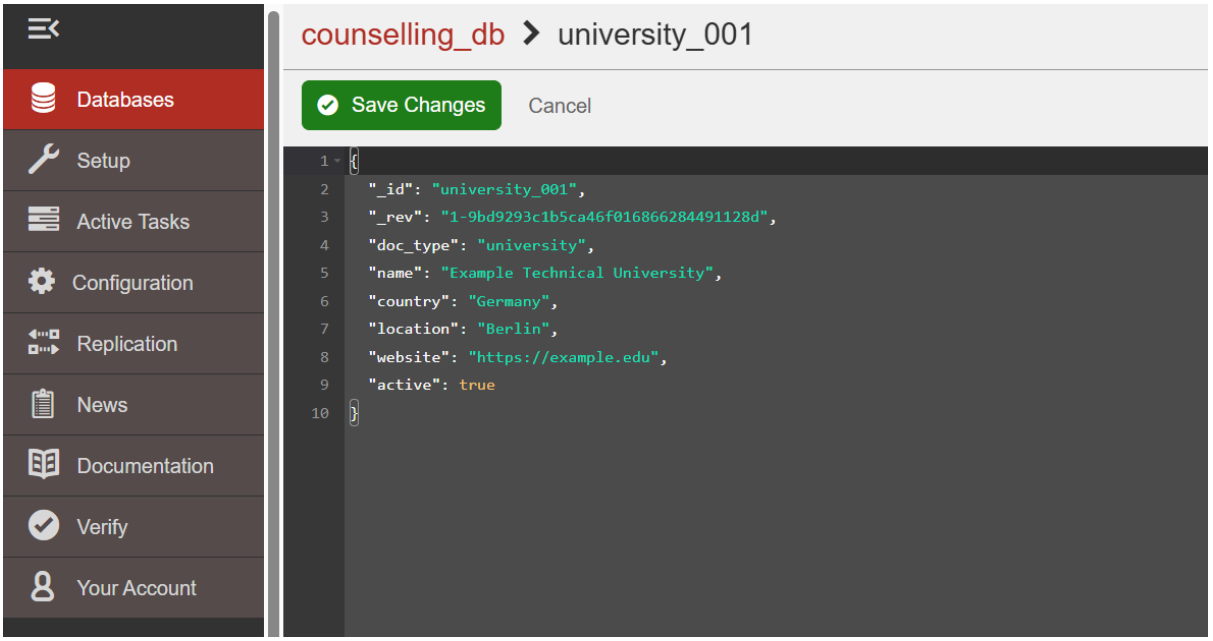


Figure 4: Fauxton JSON editor displaying university_001 document schema, containing institution details, location, and operational status.

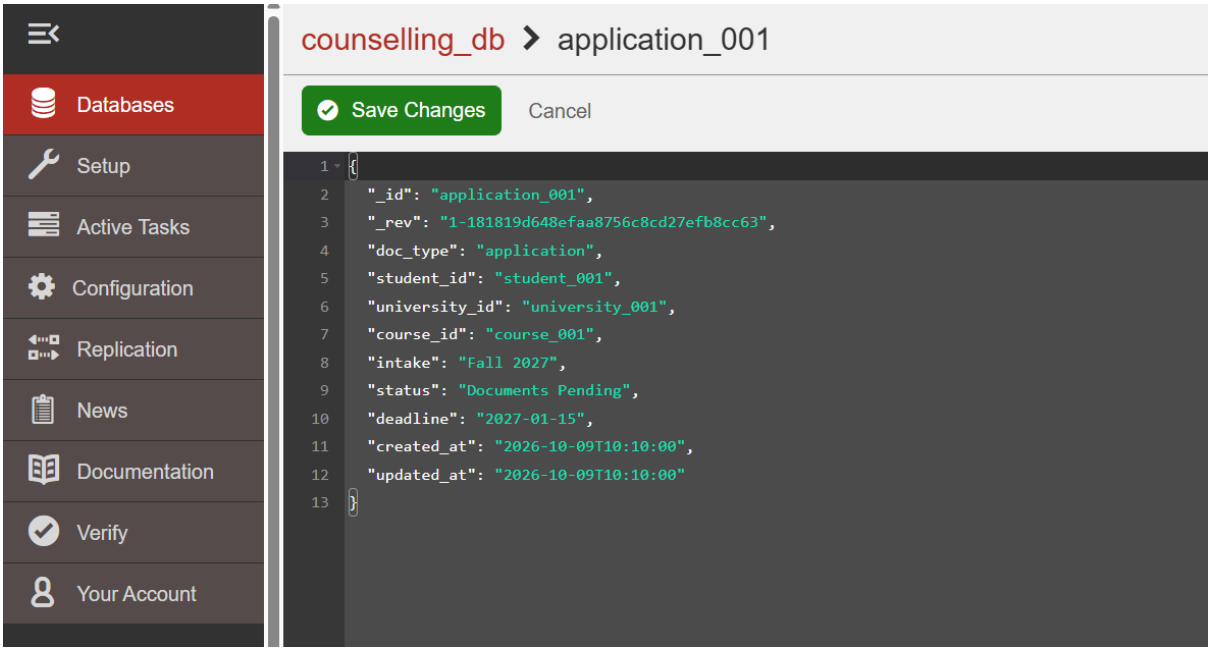


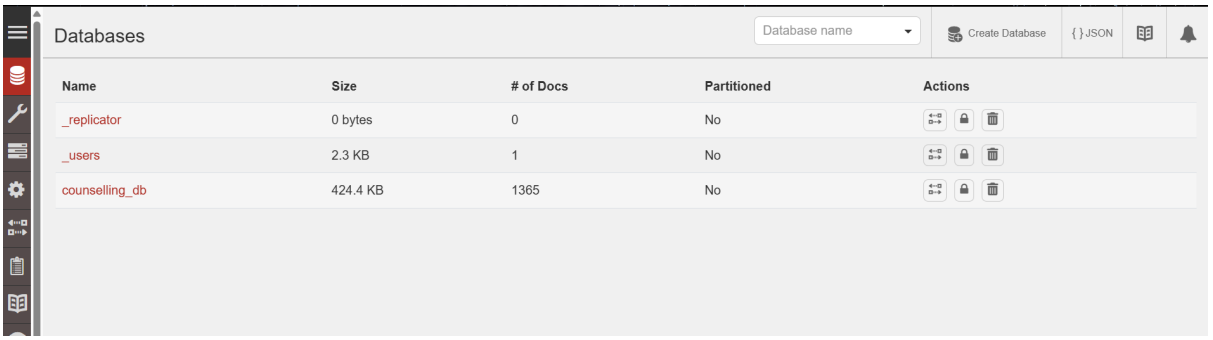
Figure 5: Fauxton JSON editor displaying application_001 record, establishing cross-document references between student, university, and course entities.

1.3 Sample Data and Record Count

Document type	count
student	20
counsellor	15
university	100
course	1155
application	40
document	10
counselling_session	25
Total	1365

The large initial dataset for university (100) and course (1155) entities represents pre-populated global institutional catalogs and standard academic course offerings required at launch. Conversely, operational entities such as student, counsellor, application, document, and counselling_session start with smaller baseline numbers reflecting initial onboarding and

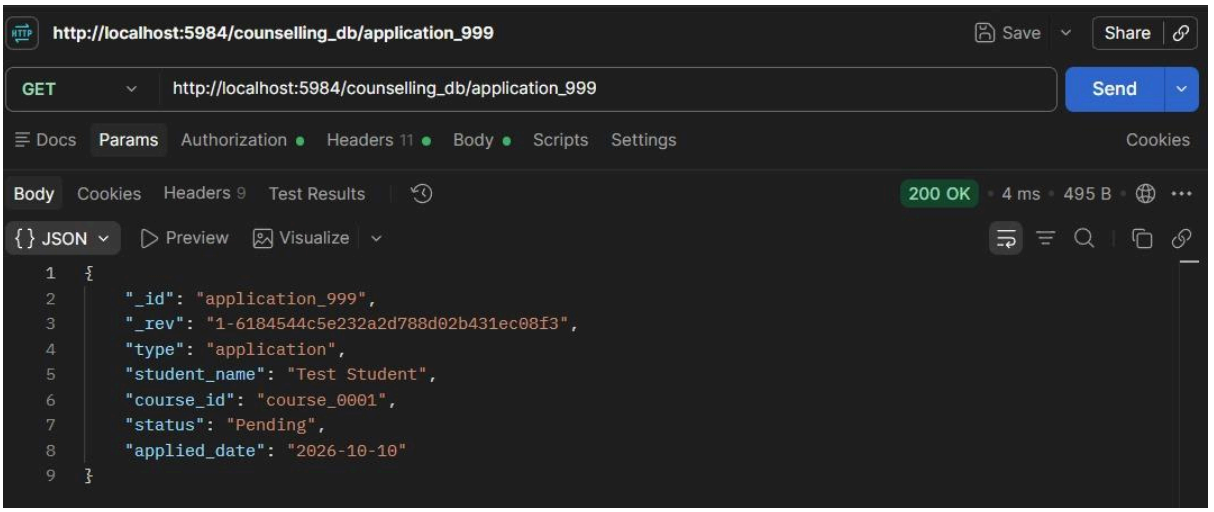
testing, which will scale dynamically during active user adoption and ongoing system utilization.



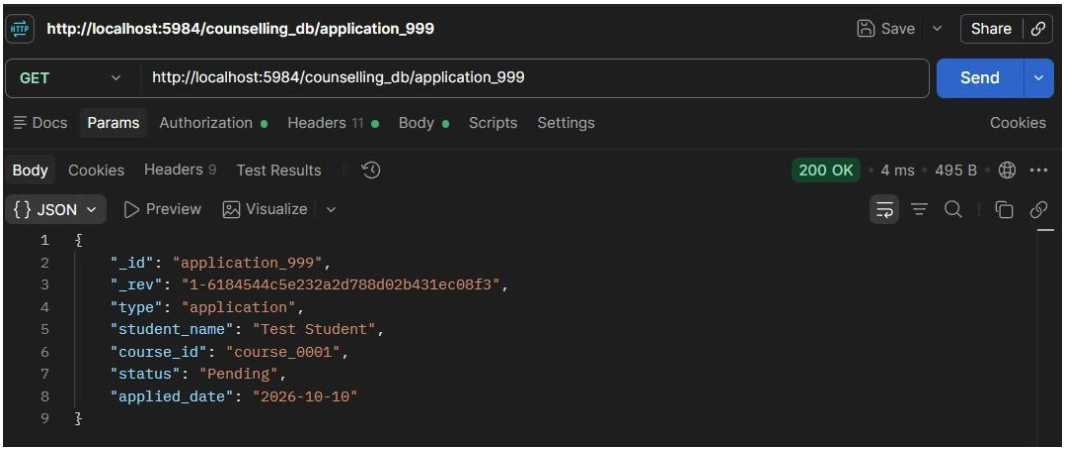
1.4 CRUD Verification

To verify complete CRUD operations via HTTP REST endpoints using Postman against CouchDB's REST API, the following sequence was executed on the `counselling_db` database:

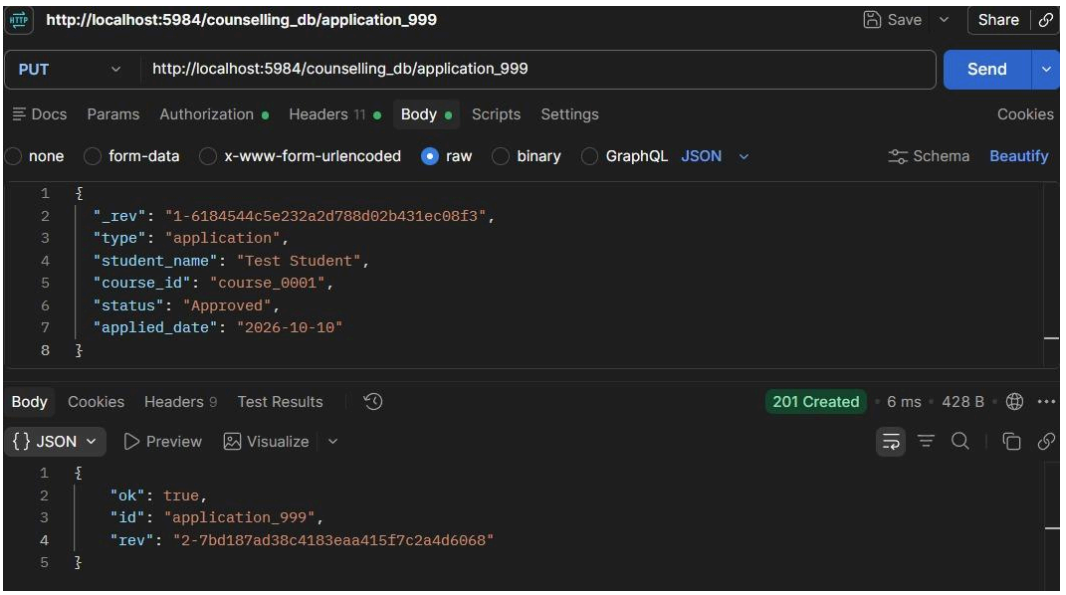
- **Create (POST / PUT):** A new application document with ID `application_999` was created using a HTTP PUT request to `http://127.0.0.1:5984/counselling_db/application_999`. CouchDB responded with a HTTP 201 Created status.



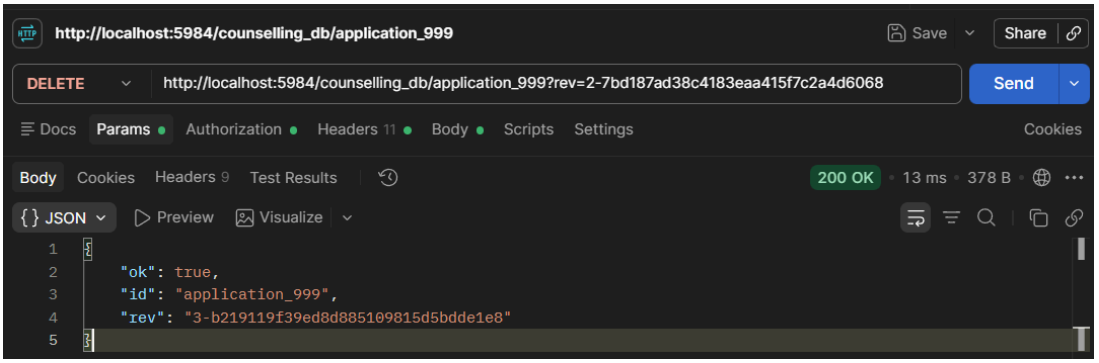
- **Read (GET):** The document was retrieved by performing a GET request to `http://127.0.0.1:5984/counselling_db/application_999`. The server returned the full document payload including system metadata (`_id` and current `_rev`) alongside custom document attributes with a HTTP 200 OK status.



- **Update (PUT):** To update the document field (e.g., updating application status to `"Verified"`), a PUT request was submitted to `http://127.0.0.1:5984/counselling_db/application_999` with the updated payload specifying the current `_rev` value. CouchDB successfully accepted the change and incremented the revision token with a HTTP 201 Created status.



- **Delete (DELETE):** A disposable test document (`application_999`) was deleted via a DELETE request specifying the exact current revision query parameter. CouchDB marked the document as deleted with a HTTP 200 OK status.



1.5 Indexes and Views

Mango JSON index

A **Mango JSON index** in Apache CouchDB is a secondary index created over specified JSON fields to accelerate declarative document queries executed via the CouchDB `_find` endpoint (Mango query engine).

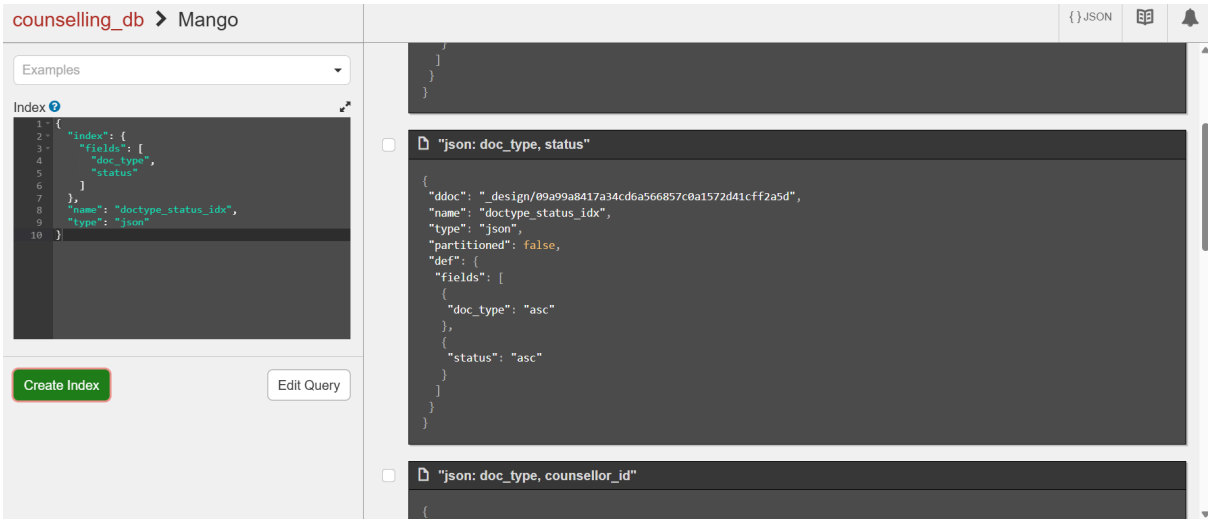


Figure 6: Fauxton interface displaying the creation and configuration of the Mango JSON index (doctype_status_idx) over the doc_type and status fields.

CouchDB view (design document)

CouchDB views are defined in design documents using map functions. This example emits application documents keyed by status.

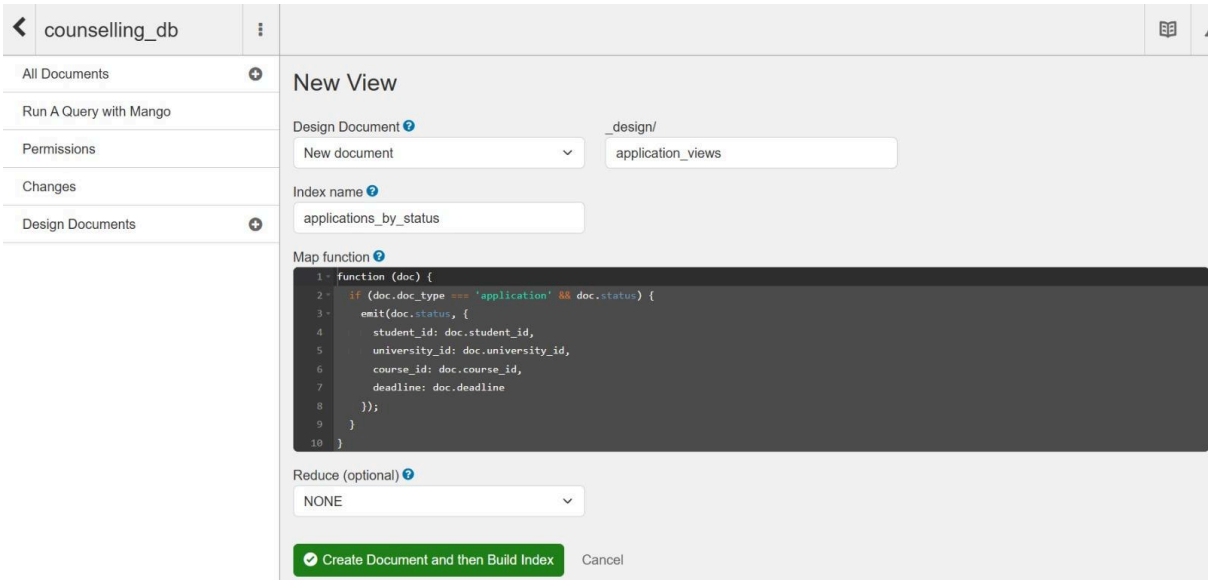


Figure 7: Creation of the CouchDB design document (application_views) defining the applications_by_status map function.

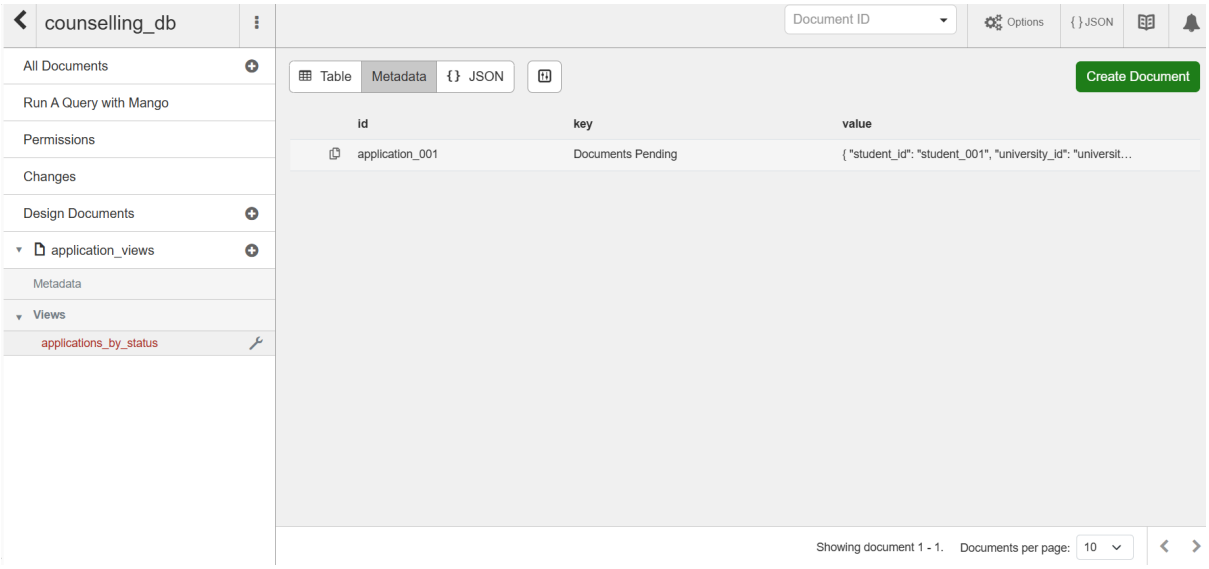


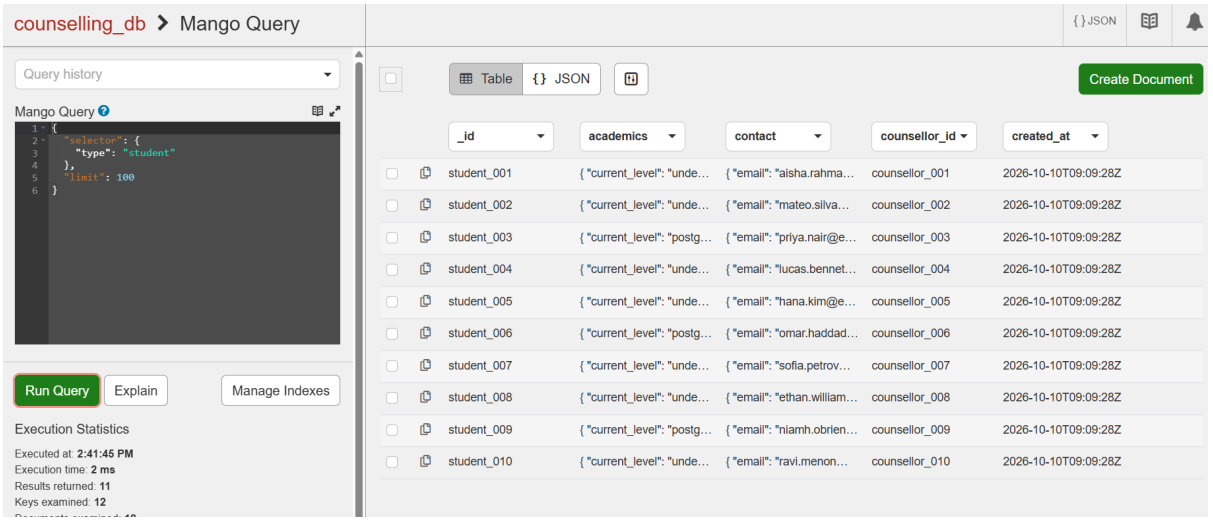
Figure 8: Fauxton view execution results displaying application documents indexed and emitted by their application status key.

2. NoSQL Queries and Transactions

2.1 Queries

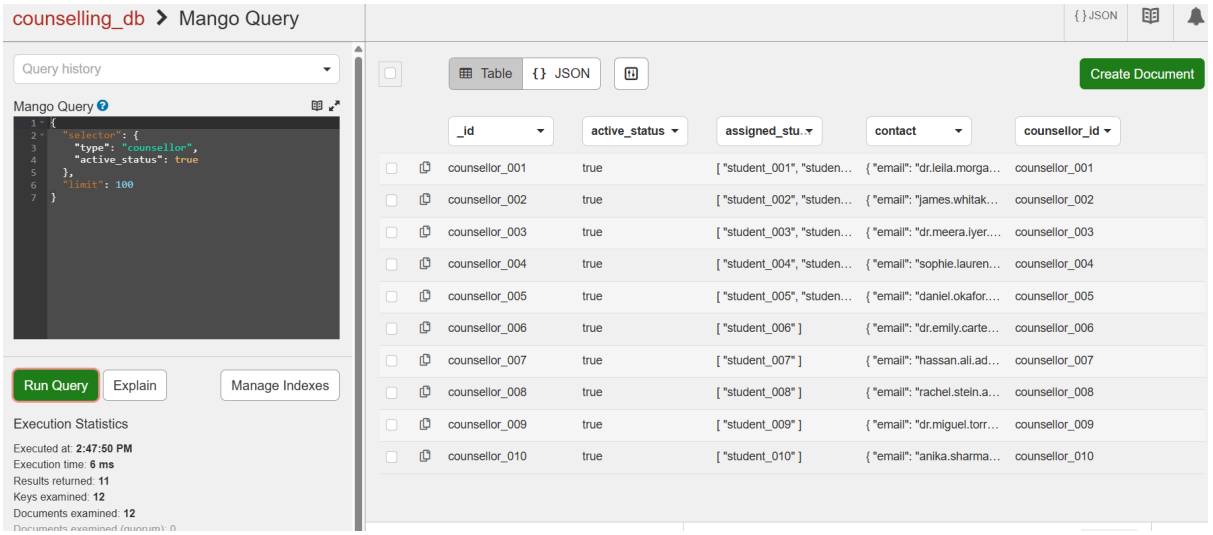
1.Get all students

```
{
  "selector": {
    "type": "student"
  },
  "limit": 100
}
```



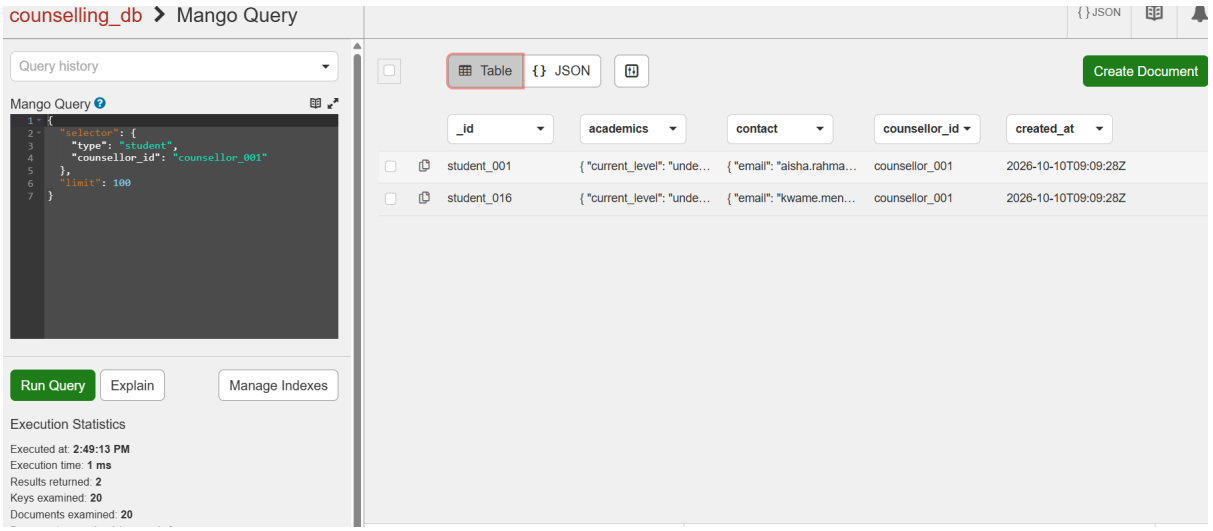
2.Get all active counsellors

```
{
  "selector": {
    "type": "counsellor",
    "status": "active"
  }
}
```



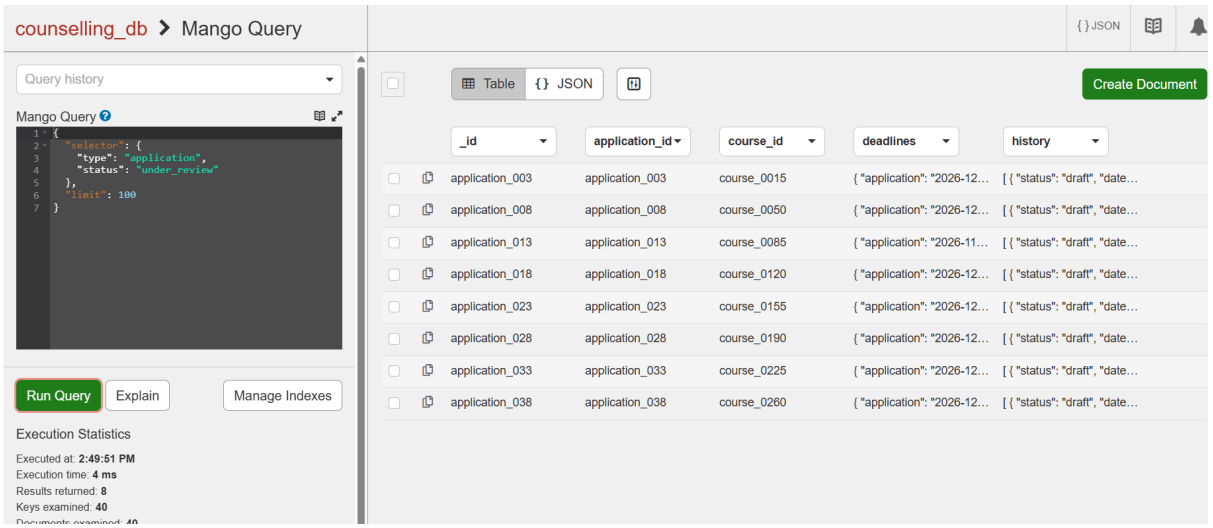
3.Find students assigned to counsellor counsellor_001

```
{
  "selector": {
    "type": "student",
    "counsellor_id": "counsellor_001"
  }
}
```



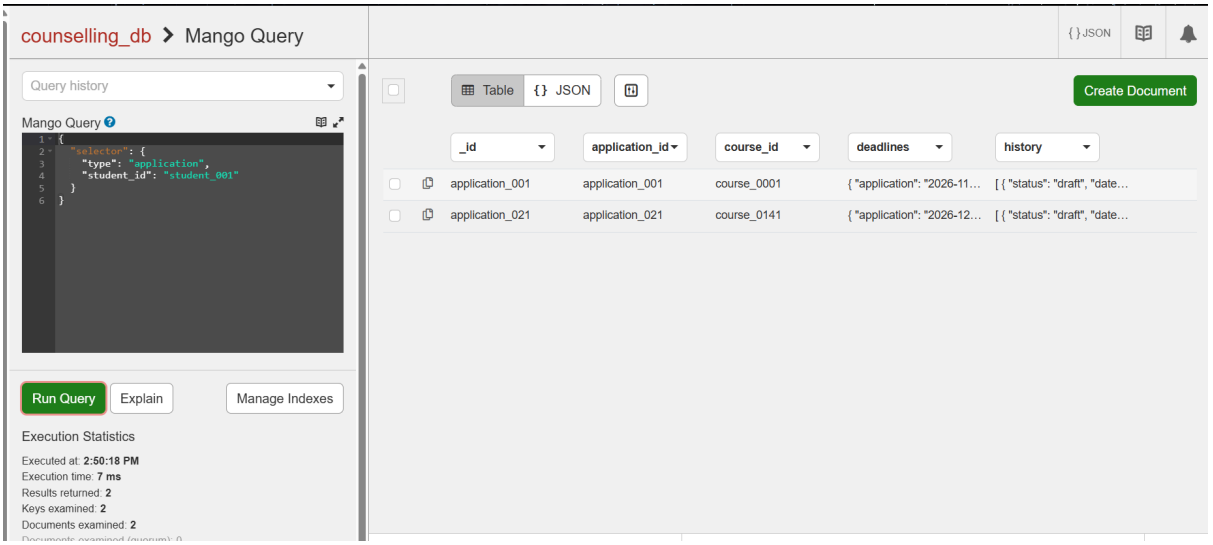
4.Get applications currently under review

```
{
  "selector": {
    "type": "application",
    "status": "under_review"
  }
}
```



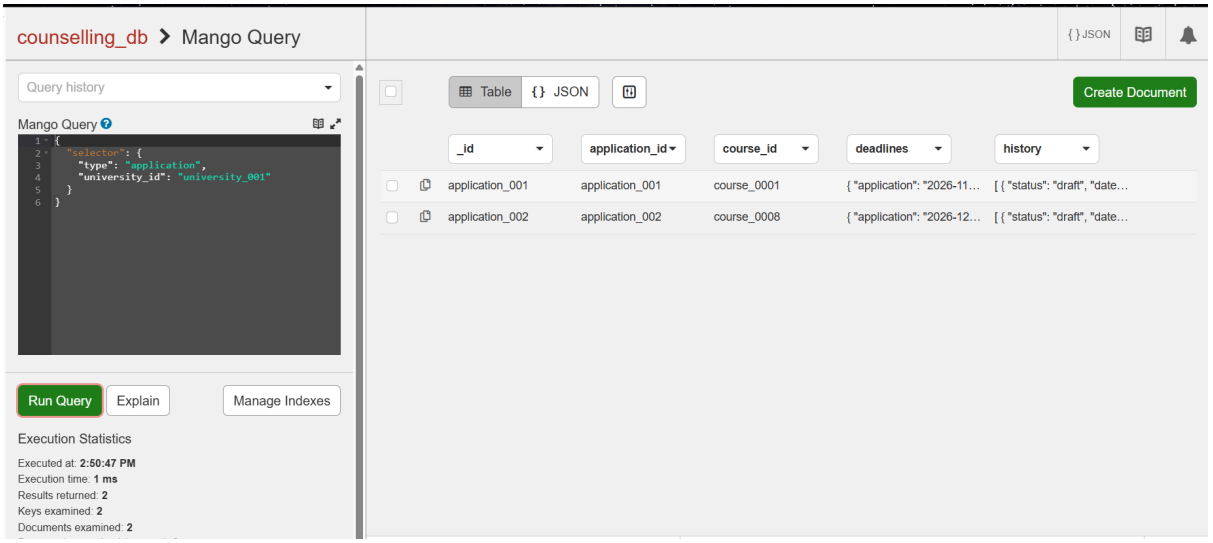
5.Get all applications for student student_001

```
{
  "selector": {
    "type": "application",
    "student_id": "student_001"
  }
}
```



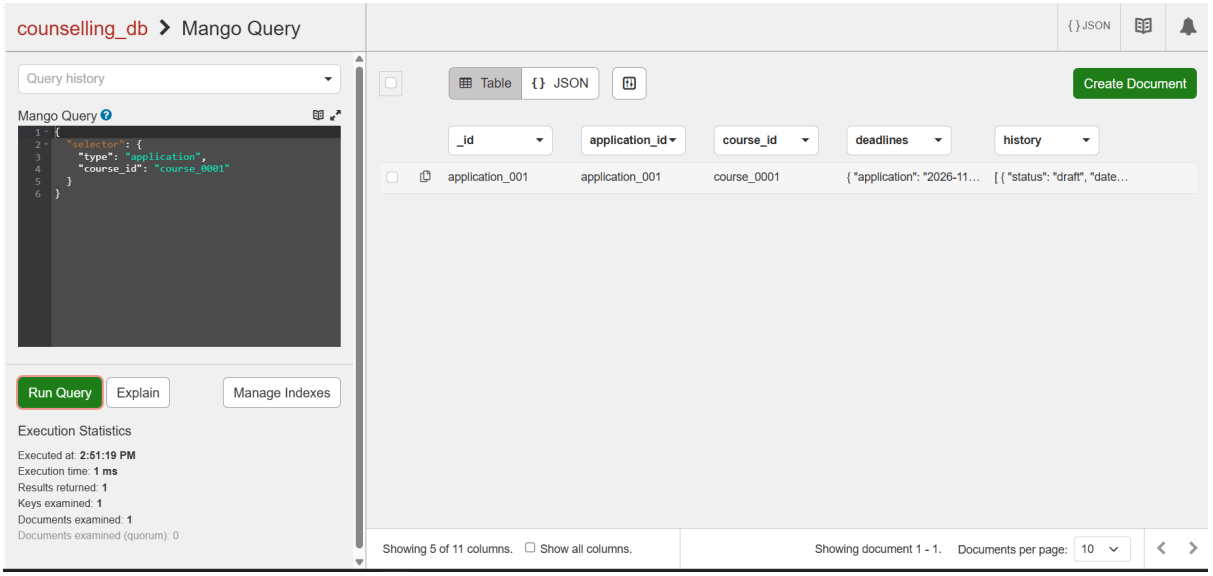
6.Get applications submitted to university_001

```
{
  "selector": {
    "type": "application",
    "university_id": "university_001"
  }
}
```



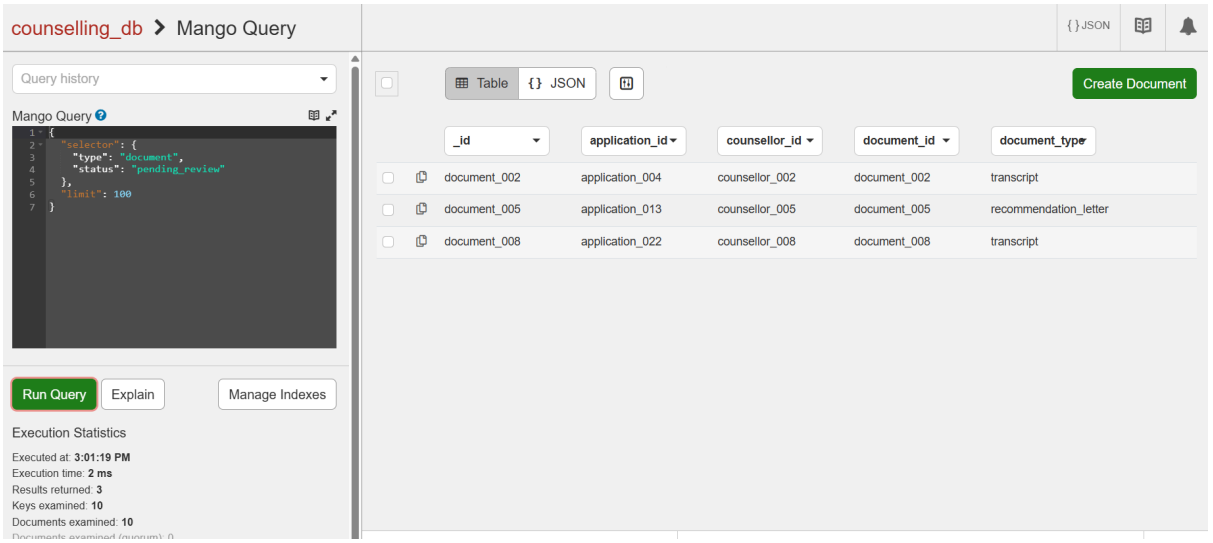
7.Get applications for course course_0001

```
{
  "selector": {
    "type": "application",
    "course_id": "course_0001"
  }
}
```



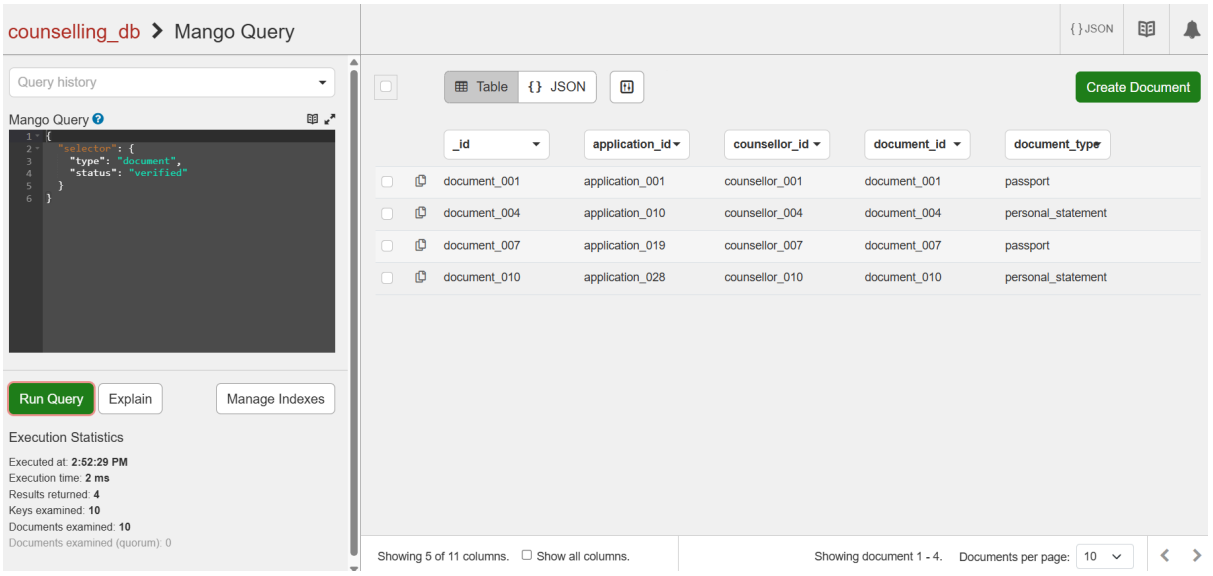
8.Get pending document reviews

```
{
  "selector": {
    "type": "document",
    "status": "pending_review"
  },
  "limit": 100
}
```



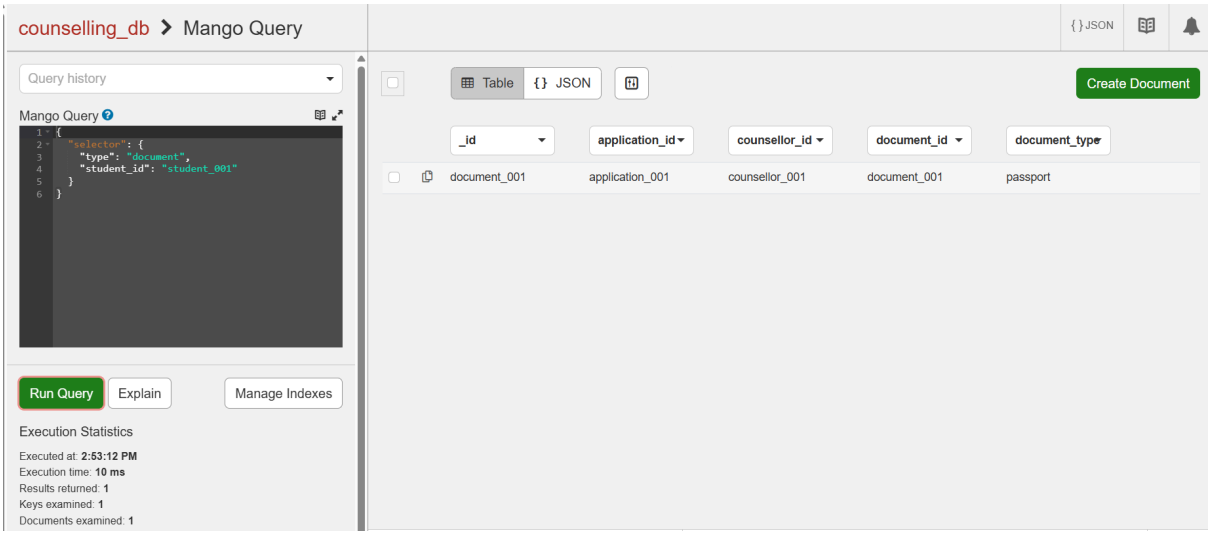
9.Get verified documents

```
{
  "selector": {
    "type": "document",
    "status": "verified"
  }
}
```



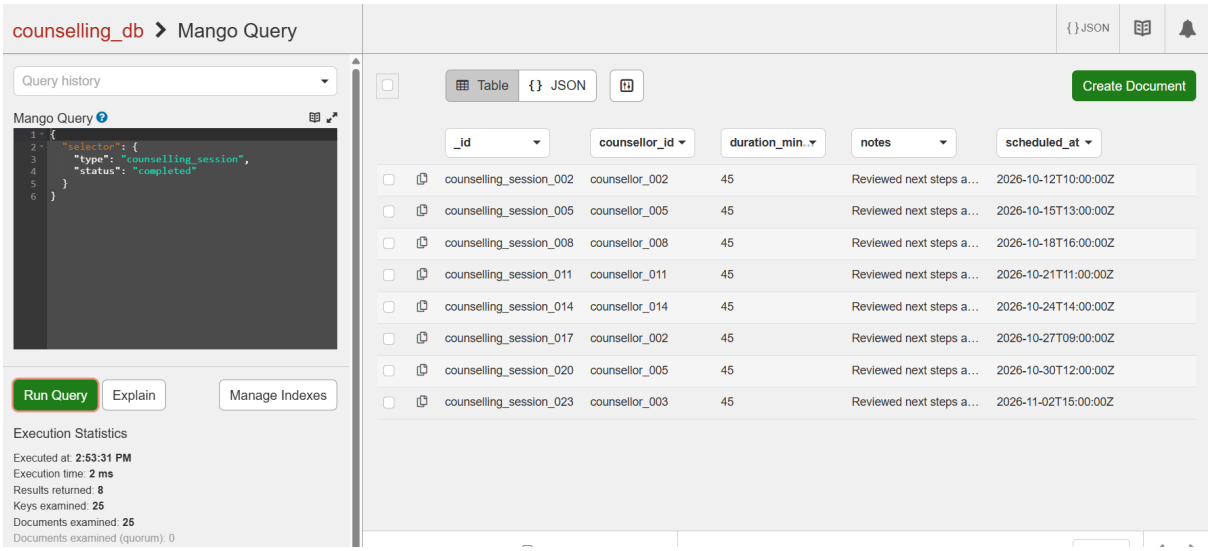
10.Get documents belonging to student_001

```
{
  "selector": {
    "type": "document",
    "student_id": "student_001"
  }
}
```



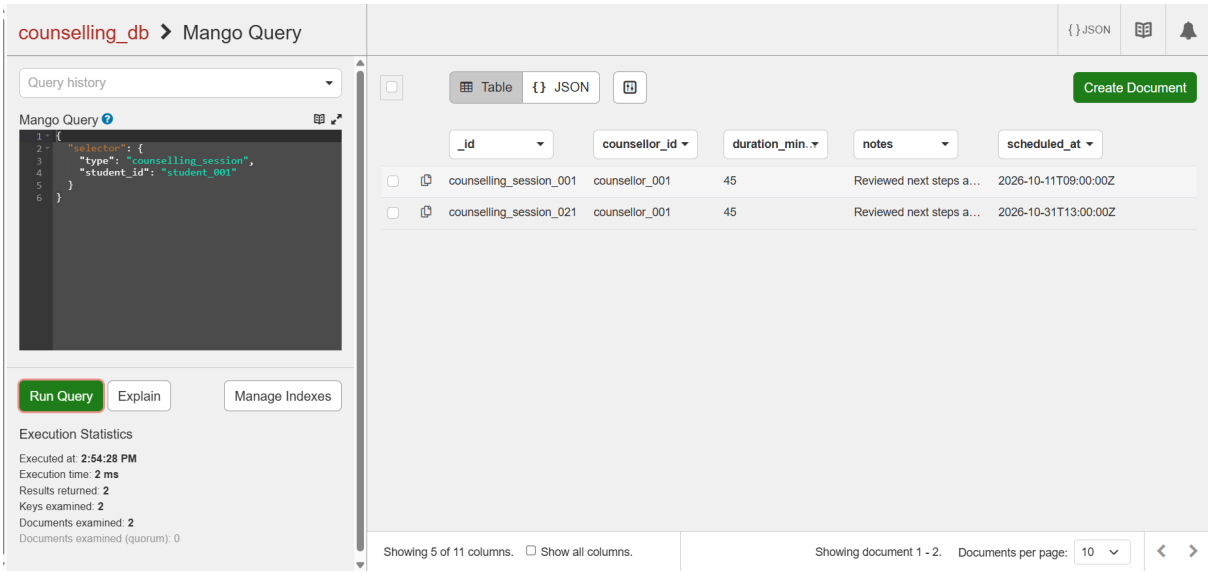
11.Get completed counselling sessions

```
{
  "selector": {
    "type": "counselling_session",
    "status": "completed"
  }
}
```



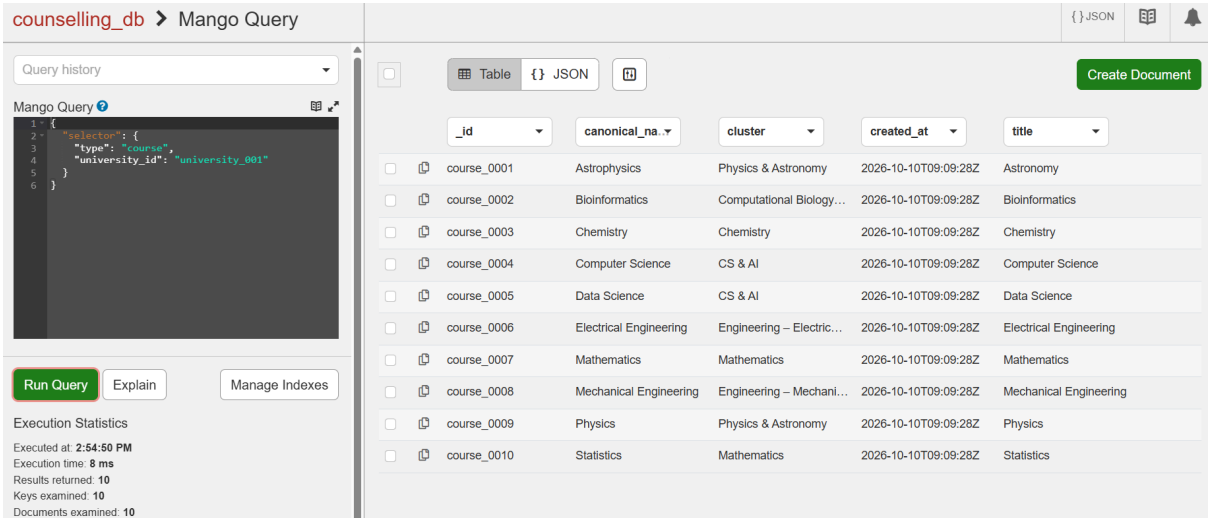
12.Get counselling sessions for student_001

```
{
  "selector": {
    "type": "counselling_session",
    "student_id": "student_001"
  }
}
```



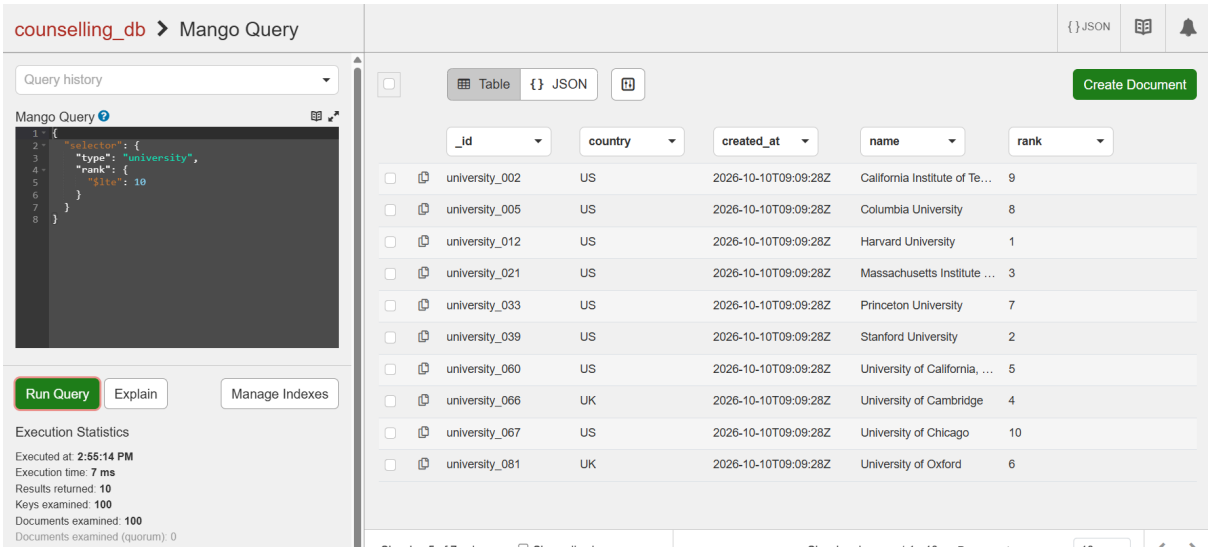
13.Get courses offered by university_001

```
{
  "selector": {
    "type": "course",
    "university_id": "university_001"
  }
}
```



14.Get universities ranked in the top 10

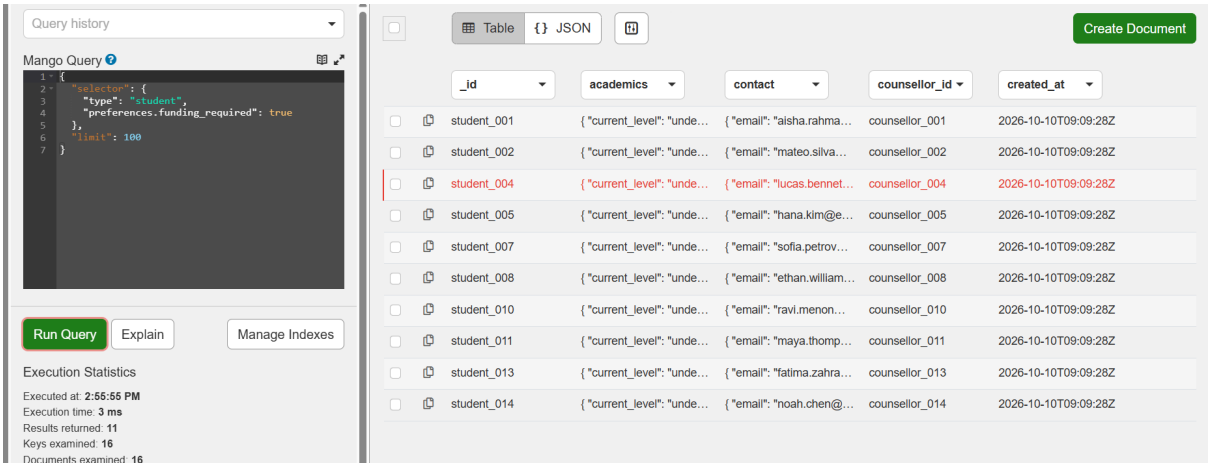
```
{
  "selector": {
    "type": "university",
    "rank": {
      "$lte": 10
    }
  }
}
```



15.Find students requiring funding

```
{
 "selector": {
    "type": "student",
    "preferences.funding_required": true
```

```
  },
  "limit": 100
}
```



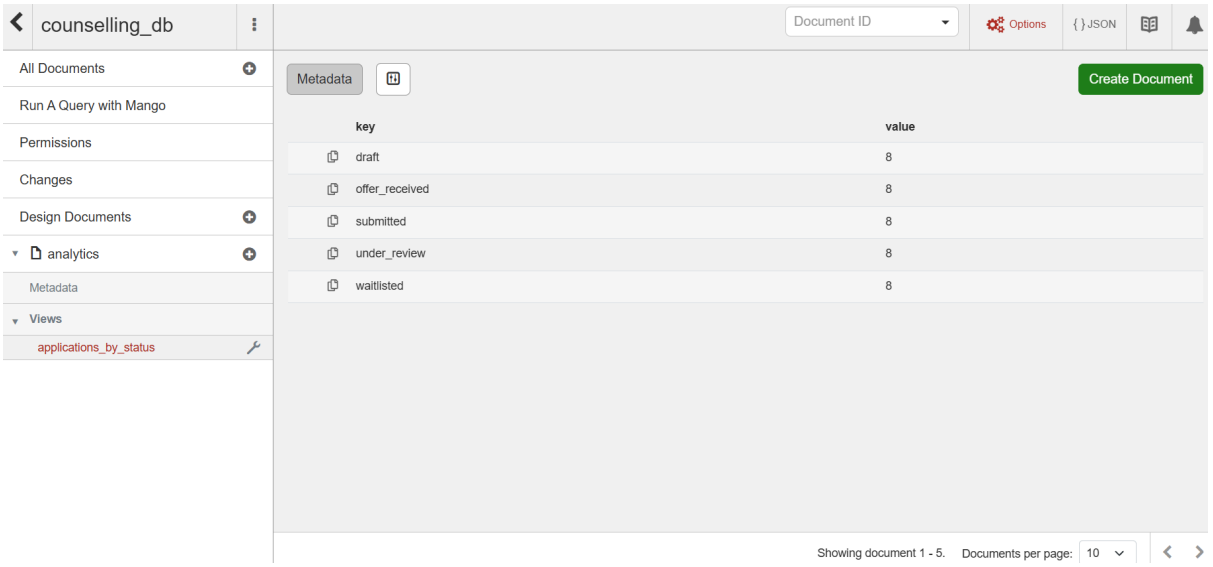
2.2 Cross-document lookup and aggregation

Mango does not perform SQL joins across independent documents. For a student–application–course–university workflow, query applications first, then fetch related documents using their IDs. Describe this as multi-step document lookup, not a native SQL JOIN.

For grouped summaries, use a CouchDB map/reduce view. Example: count applications by status.

```
function (doc) {
  if (doc.type === "application" && doc.status) {
    emit(doc.status, 1);
  }
}
```

Run the view with group=true and capture the actual count for each status. This is view-based aggregation.



2.3 Transaction handling and revision conflicts

CouchDB provides atomic writes at the individual-document level. Updates use revision tokens through the _rev field, which provides optimistic concurrency control. CouchDB does not provide a native ACID transaction spanning the application document and its application-history document.